

Smart Interview Trainer

Product Requirements Document and Software Engineering Blueprint

Goal: Build a web application that helps students prepare for job interviews by practicing questions, answering by text or voice, receiving feedback, and tracking progress over time.

Project Type: Web Application

Main Users: Students, junior developers, internship applicants

MVP Focus: Interview practice, voice-to-text answers, feedback, dashboard

Prepared For: Student Software Engineering Project

June 15, 2026

Contents

1	Project Idea	3
1.1	Problem Being Solved	3
1.2	Target Users	3
1.3	Value Proposition	3
1.4	Main Objectives	4
1.5	Competitor Analysis	4
1.6	Why This Application Is Useful	4
2	Requirements Analysis	5
2.1	Functional Requirements	5
2.2	Non-Functional Requirements	5
3	User Roles	6
4	User Stories	6
5	Use Cases	7
5.1	Use Case: Register Account	7
5.2	Use Case: Login	8
5.3	Use Case: Start Interview	8
5.4	Use Case: Answer Question by Voice	8
6	Wireframe Planning	8
6.1	Landing Page	8
6.2	Register Page	9
6.3	Dashboard	9
6.4	Interview Session Page	9
6.5	Admin Panel	10
7	System Architecture	10
7.1	Architecture Overview	10
7.2	Components	10
7.3	Simple Architecture Diagram	10
8	Technology Stack Selection	11
9	Database Design	11
9.1	Main Tables	11
9.2	SQL Table Definitions	11
10	API Design	12
10.1	Authentication Endpoints	12
10.2	Interview Endpoints	13
10.3	Admin Endpoints	13

11 Feature Breakdown	13
11.1 MVP: Version 1	13
11.2 Version 2	14
11.3 Version 3	14
12 Development Roadmap	14
13 Task Board	15
13.1 Frontend Tasks	15
13.2 Backend Tasks	16
13.3 Database Tasks	16
14 Testing Strategy	16
14.1 Testing Types	16
14.2 Example Test Cases	17
15 Security Design	17
16 Deployment Strategy	17
16.1 Development Environment	17
16.2 Production Environment	18
16.3 CI/CD Pipeline	18
17 Documentation Plan	18
18 Risks and Challenges	18
19 Future AI Features	19
20 Final Project Blueprint	19
20.1 Final MVP Goal	19
20.2 Recommended Development Order	20
20.3 Success Criteria	20

1 Project Idea

1.1 Problem Being Solved

Many students want to prepare for job interviews but do not know which questions to practice, how good their answers are, what topics they are weak in, or how they can improve over time.

Most interview preparation is unstructured. Students usually search random questions online, answer them alone, and do not receive useful feedback.

Key Point

Smart Interview Trainer solves this problem by giving users a structured way to practice interviews, receive feedback, and track their progress.

1.2 Target Users

- Medieninformatik students
- Computer science students
- Junior developers
- Working students
- Internship applicants
- Career changers
- University tutors and mentors

1.3 Value Proposition

The application helps users prepare for interviews in a practical and measurable way.

Users can:

- choose interview categories,
- answer questions by text or voice,
- receive feedback and scores,
- track progress,
- identify weak topics,
- improve over time.

1.4 Main Objectives

1. Build a simple and useful interview training platform.
2. Allow users to practice technical and HR questions.
3. Support voice-to-text answering.
4. Store user answers and scores.
5. Show progress through a dashboard.
6. Keep the project expandable for future AI and machine learning features.

1.5 Competitor Analysis

Competitor	Strength	Weakness
LeetCode	Strong coding practice	Mostly focused on algorithms
HackerRank	Technical assessments	Less focus on spoken answers
Pramp	Mock interviews	Requires another person
ChatGPT	Flexible feedback	Not structured as a progress-tracking app
YouTube tutorials	Easy to access	No personal feedback or tracking
Blogs	Many sample questions	Unstructured preparation

1.6 Why This Application Is Useful

This application is useful because it combines learning, practice, feedback, and progress tracking.

For a student project, it is strong because it includes:

- web development,
- databases,
- authentication,
- REST APIs,
- speech recognition,
- statistics,
- dashboard design,
- future machine learning and AI possibilities.

2 Requirements Analysis

2.1 Functional Requirements

Feature	Description
User Registration	Users can create an account with name, email, and password.
User Login	Users can log in securely and access their dashboard.
Logout	Users can end their session safely.
Password Reset	Users can request a password reset link. This can be optional for the MVP.
Category Selection	Users can choose topics such as Java, SQL, Web Development, ML, or HR Questions.
Difficulty Selection	Users can choose beginner, intermediate, or advanced questions.
Interview Session	Users can start a session with a selected category and difficulty.
Question Display	The app shows one question at a time.
Text Answering	Users can type their answers manually.
Voice Answering	Users can answer by speaking. Speech is converted to text.
Transcript Editing	Users can edit the generated transcript before submitting.
Answer Submission	The system saves the answer, score, feedback, and date.
Keyword Feedback	The MVP uses keyword matching to calculate a simple score.
Sample Answer	The user sees a good example answer after submitting.
Final Result	At the end, the user sees the total score and weak areas.
Progress Tracking	The app tracks performance over time.
Statistics Dashboard	The dashboard shows scores, history, and weak categories.
Profile Management	Users can update name, email, and password.
Admin Panel	Admins can manage questions, categories, and users.

2.2 Non-Functional Requirements

Requirement	Description
Performance	Dashboard should load under 2 seconds. Normal API responses should be under 500 ms.
Security	Passwords must be hashed. Routes must be protected. Inputs must be validated.
Scalability	The system should support more users, questions, and future AI features.
Usability	The interface should be simple, clear, and student-friendly.

Accessibility	The app should support keyboard navigation, labels, readable fonts, and good contrast.
Reliability	Answers should not be lost after submission. Errors should be handled clearly.
Maintainability	Code should be clean, modular, and documented.

3 User Roles

Role	Permissions	Actions
Guest	View landing page, register, login	Read app info, create account, log in
Registered User	Practice interviews and view progress	Start sessions, answer questions, use voice feature, view dashboard
Admin	Manage platform content	Create, edit, and delete questions and categories

4 User Stories

1. As a guest, I want to understand the app quickly so that I know if it helps me prepare for interviews.
2. As a guest, I want to register so that I can save my progress.
3. As a user, I want to log in so that I can access my dashboard.
4. As a user, I want to choose an interview category so that I can practice a specific topic.
5. As a user, I want to choose a difficulty level so that the questions match my skill level.
6. As a user, I want to answer questions by typing so that I can practice written explanations.
7. As a user, I want to answer questions by voice so that I can practice speaking like in a real interview.
8. As a user, I want to edit the voice transcript so that speech recognition mistakes can be corrected.
9. As a user, I want to submit my answer so that I can receive feedback.
10. As a user, I want to receive a score so that I know how good my answer was.
11. As a user, I want to see missing keywords so that I know what I forgot.
12. As a user, I want to see a sample answer so that I can learn a better explanation.
13. As a user, I want to complete an interview session so that I can simulate real preparation.
14. As a user, I want to see my final result so that I understand my performance.

15. As a user, I want to see my weak topics so that I know what to practice next.
16. As a user, I want to view my progress over time so that I stay motivated.
17. As a user, I want to see my best category so that I know my strengths.
18. As a user, I want to see my interview history so that I can compare past results.
19. As a user, I want to update my profile so that my account information stays correct.
20. As a user, I want to change my password so that my account stays secure.
21. As an admin, I want to create questions so that users can practice more topics.
22. As an admin, I want to edit questions so that mistakes can be fixed.
23. As an admin, I want to delete questions so that bad content is removed.
24. As an admin, I want to manage categories so that the app stays organized.
25. As an admin, I want to view user activity so that I understand app usage.
26. As a user, I want to practice HR questions so that I can prepare for non-technical interviews.
27. As a user, I want to practice SQL questions so that I can improve database knowledge.
28. As a user, I want the app to recommend weak topics so that I know what to study.
29. As a user, I want the app to work on mobile so that I can practice anywhere.
30. As a user, I want clear error messages so that I understand what went wrong.

5 Use Cases

5.1 Use Case: Register Account

Goal	Create a new user account.
Actor	Guest user
Preconditions	User is not logged in.
Main Flow	User opens register page, enters data, system validates input, hashes password, saves user, and redirects to login or dashboard.
Alternative Flow	If email already exists, system shows an error message.
Postconditions	New user account exists.

5.2 Use Case: Login

Goal	Authenticate user.
Actor	Registered user
Preconditions	User has an account.
Main Flow	User enters email and password. System checks credentials and returns JWT token.
Alternative Flow	If credentials are wrong, system shows an error message.
Postconditions	User is logged in.

5.3 Use Case: Start Interview

Goal	Start a new interview session.
Actor	Registered user
Preconditions	User is logged in.
Main Flow	User chooses category and difficulty. System creates session and loads questions.
Alternative Flow	If no questions exist, system shows a message.
Postconditions	Interview session is active.

5.4 Use Case: Answer Question by Voice

Goal	Submit a spoken answer.
Actor	Registered user
Preconditions	Interview session is active. Browser supports speech recognition.
Main Flow	User starts voice recording. Browser converts speech to text. User edits transcript and submits. System evaluates and saves answer.
Alternative Flow	If microphone permission is denied, user can type the answer manually.
Postconditions	Voice answer transcript is saved.

6 Wireframe Planning

6.1 Landing Page

Purpose: Explain the app and motivate users to register.

Main components:

- Logo and navigation
- Hero section
- Feature cards
- How it works section
- Call-to-action button
- Footer

6.2 Register Page

Purpose: Create a new account.

Components:

- Name input
- Email input
- Password input
- Confirm password input
- Create account button
- Link to login

6.3 Dashboard

Purpose: Show user progress and entry point to interviews.

Components:

- Welcome message
- Average score card
- Completed interviews card
- Best category card
- Weakest category card
- Recent interview list
- Start interview button

6.4 Interview Session Page

Purpose: Let the user answer interview questions.

Components:

- Question card
- Category badge
- Difficulty badge
- Text answer area
- Start voice button
- Stop voice button
- Transcript area
- Submit button
- Progress indicator

6.5 Admin Panel

Purpose: Manage questions and categories.

Components:

- Question table
- Add question form
- Edit button
- Delete button
- Category management

7 System Architecture

7.1 Architecture Overview

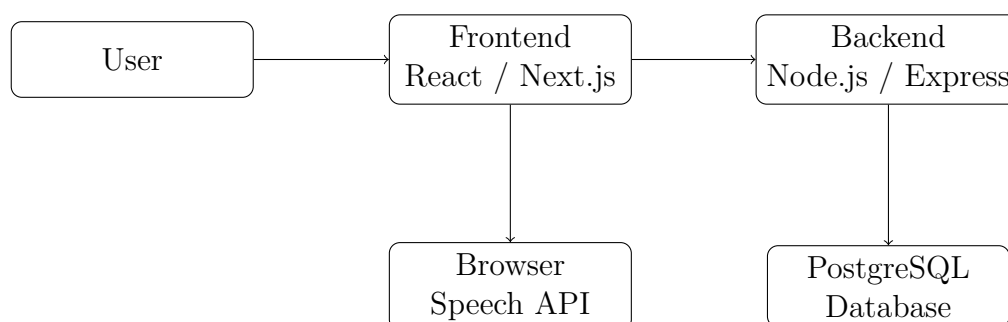
Key Point

The system follows a classic frontend-backend-database architecture. The frontend communicates with the backend through REST APIs. The backend stores and reads data from PostgreSQL.

7.2 Components

Component	Responsibility
Frontend	UI, forms, routing, voice-to-text, API calls
Backend	Authentication, business logic, scoring, progress calculation
Database	Store users, questions, sessions, answers, and scores
Authentication	JWT-based login and route protection
Speech Recognition	Browser speech-to-text API
Deployment	Vercel for frontend, Railway for backend and database

7.3 Simple Architecture Diagram



8 Technology Stack Selection

Layer	Technology	Reason
Frontend	React, Next.js, TypeScript	Modern, component-based, scalable, good routing
Backend	Node.js, Express	Simple REST API development and good for students
Database	PostgreSQL	Reliable relational database with strong SQL support
Authentication	JWT	Common for modern web applications
Deployment	Vercel, Railway	Simple deployment for student projects
Containerization	Docker	Makes backend easier to deploy consistently

Important

Alternative backend: Java Spring Boot is also a good choice if the student wants to focus more on Java and enterprise backend development. However, Node.js with Express is simpler for a first full-stack MVP.

9 Database Design

9.1 Main Tables

Table	Purpose
users	Stores user accounts and roles
categories	Stores interview topics
questions	Stores interview questions and expected keywords
interview_sessions	Stores interview attempts
answers	Stores submitted answers, scores, and feedback

9.2 SQL Table Definitions

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(20) DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE categories (
```

```
id SERIAL PRIMARY KEY,  
name VARCHAR(100) NOT NULL UNIQUE,  
description TEXT  
);  
  
CREATE TABLE questions (  
id SERIAL PRIMARY KEY,  
category_id INT NOT NULL,  
difficulty VARCHAR(30) NOT NULL,  
question_text TEXT NOT NULL,  
sample_answer TEXT NOT NULL,  
expected_keywords TEXT NOT NULL,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (category_id) REFERENCES categories(id)  
);  
  
CREATE TABLE interview_sessions (  
id SERIAL PRIMARY KEY,  
user_id INT NOT NULL,  
category_id INT NOT NULL,  
difficulty VARCHAR(30) NOT NULL,  
final_score INT,  
started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
ended_at TIMESTAMP,  
FOREIGN KEY (user_id) REFERENCES users(id),  
FOREIGN KEY (category_id) REFERENCES categories(id)  
);  
  
CREATE TABLE answers (  
id SERIAL PRIMARY KEY,  
user_id INT NOT NULL,  
question_id INT NOT NULL,  
session_id INT NOT NULL,  
answer_text TEXT NOT NULL,  
score INT NOT NULL,  
feedback TEXT,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (user_id) REFERENCES users(id),  
FOREIGN KEY (question_id) REFERENCES questions(id),  
FOREIGN KEY (session_id) REFERENCES interview_sessions(id)  
);
```

10 API Design

10.1 Authentication Endpoints

Method	Route	Description
POST	/api/auth/register	Create a new user account

POST	/api/auth/login	Log in and receive JWT token
GET	/api/auth/me	Get current logged-in user

10.2 Interview Endpoints

Method	Route	Description
GET	/api/categories	Get all interview categories
GET	/api/questions	Get questions by category and difficulty
POST	/api/interviews/start	Start a new interview session
POST	/api/interviews/submit	Submit answer and receive feedback
POST	/api/interviews/finish	Finish interview and calculate final score
GET	/api/progress	Get user progress summary
GET	/api/progress/history	Get score history

10.3 Admin Endpoints

Method	Route	Description
POST	/api/admin/questions	Create question
PUT	/api/admin/questions/:id	Update question
DELETE	/api/admin/questions/:id	Delete question

11 Feature Breakdown

11.1 MVP: Version 1

MVP Focus

Version 1 should stay realistic. The focus is not advanced AI. The focus is a working app with interview practice, voice-to-text, feedback, and progress tracking.

- Landing page
- Register and login
- JWT authentication
- Dashboard
- Category selection
- Difficulty selection
- Interview session

- Text answers
- Voice-to-text answers
- Keyword-based scoring
- Feedback page
- Sample answer
- Progress dashboard
- Admin question management

11.2 Version 2

- Adaptive interviews
- Weakness detector
- Recommended practice
- Better statistics
- KNN-based recommendation system
- Decision tree performance prediction

11.3 Version 3

- AI feedback
- AI-generated questions
- AI interviewer follow-up questions
- Resume-based interview generation
- Voice interview simulation
- Text-to-speech interviewer

12 Development Roadmap

Week	Goals	Deliverables	Hours
1	Planning and setup	GitHub repo, frontend setup, backend setup, database setup	10
2	Authentication	Register, login, JWT, protected routes	12

3	Database and questions	Schema, seed data, question API	10
4	Interview session	Setup page, question display, text answers	12
5	Scoring and feedback	Keyword scoring, feedback page, saved answers	12
6	Voice answer feature	Speech-to-text, transcript editing, voice submit	10
7	Dashboard and progress	Statistics, history, weak categories	12
8	Admin panel and testing	Admin CRUD, bug fixes, tests	12
9	Deployment	Vercel, Railway, documentation	8

13 Task Board

13.1 Frontend Tasks

- Create landing page
- Create register page
- Create login page
- Create dashboard
- Create interview setup page
- Create interview session page
- Create feedback page
- Create progress page
- Create profile page
- Create admin panel
- Add voice button
- Add transcript text area
- Add loading states
- Add error messages
- Make layout responsive

13.2 Backend Tasks

- Set up Express server
- Connect PostgreSQL
- Create auth routes
- Create JWT middleware
- Create user controller
- Create category controller
- Create question controller
- Create interview controller
- Create evaluation service
- Create progress service
- Create admin middleware

13.3 Database Tasks

- Create users table
- Create categories table
- Create questions table
- Create interview sessions table
- Create answers table
- Add foreign keys
- Add seed categories
- Add seed questions

14 Testing Strategy

14.1 Testing Types

Testing Type	Purpose
Unit Testing	Test scoring, keyword matching, validation, and utility functions
Integration Testing	Test register, login, interview start, answer submit, and database interaction
API Testing	Test endpoints using Postman or Insomnia

UI Testing	Test forms, buttons, layout, voice feature, and error messages
User Acceptance Testing	Let another student try the app and give feedback

14.2 Example Test Cases

Test Case	Input	Expected Result
Register user	Valid name, email, password	User account is created
Login user	Correct credentials	JWT token is returned
Start interview	Category and difficulty	Session is created
Submit answer	Answer with keywords	Score and feedback returned
Voice answer	Spoken answer	Transcript appears in text field
Admin creates question	Valid question data	Question is saved

15 Security Design

Security Area	Solution
Authentication	Use JWT tokens
Authorization	Protect user and admin routes
Password Hashing	Use bcrypt
Input Validation	Validate all request bodies
Rate Limiting	Limit repeated login attempts
SQL Injection	Use parameterized queries or ORM
XSS Protection	Escape user-generated content
CSRF Protection	Prefer secure cookies later; document JWT tradeoff in MVP

16 Deployment Strategy

16.1 Development Environment

Tools:

- VS Code
- Node.js
- PostgreSQL
- Git
- Postman or Insomnia

16.2 Production Environment

Part	Deployment Platform
Frontend	Vercel
Backend	Railway
Database	Railway PostgreSQL
Code Repository	GitHub

16.3 CI/CD Pipeline

Simple flow:

1. Developer pushes code to GitHub.
2. Vercel deploys frontend automatically.
3. Railway deploys backend automatically.
4. Environment variables are configured in deployment platforms.
5. Final app is tested in production.

17 Documentation Plan

The project should include:

- README
- Requirements Specification
- Architecture Document
- Database Documentation
- API Documentation
- User Manual
- Developer Guide
- Deployment Guide

18 Risks and Challenges

Risk	Problem	Mitigation
Voice recognition issue	Not all browsers support it well	Provide text input fallback

Project too large	Too many features can slow progress	Focus on MVP first
Simple scoring	Keyword scoring is limited	Improve later with AI
Database complexity	Relations may confuse beginners	Keep schema simple and documented
Security mistakes	Auth can be implemented incorrectly	Use standard libraries and middleware

19 Future AI Features

- Voice interview simulation
- AI interviewer
- Emotion detection
- Resume analysis
- Personalized learning paths
- AI-generated interview questions
- Job description based interview generation

Important

These AI features should not be part of the first version. They are planned for future versions after the MVP is stable.

20 Final Project Blueprint

20.1 Final MVP Goal

Build a web app where a user can:

- register,
- log in,
- choose an interview category,
- choose difficulty,
- answer questions by text or voice,
- receive keyword-based feedback,
- see a score,
- track progress,
- view dashboard statistics.

20.2 Recommended Development Order

1. Create database schema.
2. Build authentication.
3. Add categories and questions.
4. Build interview session.
5. Add answer submission.
6. Add scoring system.
7. Add feedback screen.
8. Add voice-to-text.
9. Add dashboard.
10. Add admin panel.
11. Test everything.
12. Deploy.

20.3 Success Criteria

The project is successful when:

- a new user can register,
- the user can log in,
- the user can start an interview,
- the user can answer by text,
- the user can answer by voice,
- the transcript can be edited,
- the system gives a score,
- the system gives feedback,
- the answer is saved,
- progress is visible,
- the app is deployed online.

MVP Focus

The first version should be simple, stable, and useful. Advanced AI and adaptive learning should come later after the core app works correctly.